COMPILADORES

AULA 18

Otimização local e análise de fluxo de dados

Blocos básicos • transformações locais • análises forward e backward

Conteúdos das aulas de 14/10 e 15/10

Material alinhado ao plano de ensino e à especificação MINIC

Apresentação e lugar desta aula no curso

Esta apostila dá continuidade ao percurso iniciado com a representação intermediária (IR) e a geração de código. A IR em código de três endereços torna explícitas as operações e os fluxos de controle; agora, ela será organizada em blocos e transformada por otimizações que devem preservar o comportamento observável do programa.

A análise de laços — dominadores, invariantes, movimento de código e variáveis de indução — pertence à Aula 19 (Capítulo 9, seção 9.6). A seleção de instruções, os registradores e as convenções de chamada foram tratados na Aula 17. Assim, esta apostila utiliza a IR e o modelo de execução já estudados sem antecipar esses tópicos.

Objetivos de aprendizagem

- Particionar uma sequência de IR em blocos básicos e construir seu grafo de fluxo.
- Aplicar simplificação algébrica, propagação local de constantes, eliminação de cópias e código morto com justificativa semântica.
- Definir IN/OUT, gen/kill e funções de transferência para análises forward e backward.
- Resolver, por iteração até ponto fixo, problemas de reaching definitions, available expressions e live variables.
- Explicar como a propagação global de constantes usa informação de caminhos e por que deve ser conservadora.

1. Otimização de código: princípio de corretude

Uma otimização é uma transformação do programa que busca reduzir custo — por exemplo, número de instruções, tempo de execução ou pressão sobre registradores — sem alterar os resultados definidos pela linguagem. “Mais curto” não significa automaticamente “mais rápido” nem “correto”: custo depende do alvo, e a transformação precisa respeitar tipos, efeitos e ordem de avaliação.

A especificação MINIC define tipos int, float, bool e char; vetores unidimensionais; atribuições; chamadas; condicionais e laços. Entre as conversões implícitas recomendadas estão int→float e char→int; float→int, conversões envolvendo bool e outras não são implícitas na versão estrita. O resto (%) aceita apenas inteiros, operadores lógicos exigem bool e divisor constante zero deve ser rejeitado. Essas regras limitam quais identidades são válidas.

1.1 Cuidados semânticos

- Não aplicar uma identidade fora do domínio dos tipos. Por exemplo, x * 0 só pode ser reduzido após confirmar que a operação e seus efeitos são compatíveis com a semântica MINIC.
- Não remover nem reordenar chamadas, read, acessos potencialmente inválidos a vetores ou operações que possam produzir erro observável.
- Não assumir que a aritmética de inteiros tem as propriedades matemáticas dos inteiros ilimitados: overflow e comportamento da implementação devem ser considerados.
- Respeitar curto-circuito de && e || quando a implementação da linguagem o define; a expressão à direita pode não ser avaliada.
- Evitar otimizações de memória sobre vetores quando houver escrita que possa alterar o elemento lido.

2. Blocos básicos e grafo de fluxo

Um bloco básico é uma sequência máxima de instruções de três endereços com uma única entrada e uma única saída: o controle só entra pela primeira instrução; dentro do bloco, o fluxo segue sequencialmente; desvios ou transferências de controle só podem ocorrer na última instrução. Essa propriedade permite raciocinar sobre uma sequência sem considerar desvios internos.

2.1 Como identificar líderes

Um líder é a primeira instrução de um bloco básico. Para uma lista linear de IR, marque como líder (1) a primeira instrução do procedimento; (2) qualquer destino de salto; (3) qualquer instrução imediatamente posterior a um salto condicional ou incondicional. Depois, cada bloco começa em um líder e termina imediatamente antes do próximo líder, ou na instrução final. Saltos para rótulos devem apontar para o início de bloco.

Líderes: 1 (início do procedimento), 3 (destino do salto em 7), 4 (instrução seguinte ao salto condicional em 3) e 8 (destino do salto condicional em 3). Como print é uma instrução ordinária neste exemplo, a linha 9 não inicia um novo bloco. Os blocos são B1={1–2}, B2={3}, B3={4–7}, B4={8–9}.

Bloco	Sucessores	Justificativa
B1	B2	Continuação sequencial
B2	B4 e B3	Desvio condicional: verdadeiro/falso
B3	B2	Salto incondicional para a condição
B4	EXIT	Retorno

2.2 Grafo de fluxo

Cada bloco é um nó; uma aresta B→C indica que C pode executar imediatamente depois de B. O grafo explicita caminhos possíveis, junções e ciclos. Um bloco sem sucessores pode terminar o procedimento; em implementações, nós artificiais ENTRY e EXIT facilitam definir condições de contorno para as análises.

No exemplo, o ciclo B2→B3→B2 representa o laço. O grafo não afirma que todos os caminhos são executados; registra possibilidades de fluxo de controle. Uma análise global deve considerar todos os caminhos pertinentes, inclusive os que passam por ciclos.

2.3 Exercício guiado: líderes e arestas

Considere a IR: (1) a=1; (2) if c goto 5; (3) b=a+2; (4) goto 6; (5) b=a+3; (6) print(b); (7) return 0. Os líderes são 1, 3, 5 e 6. Os blocos são B1={1–2}, B2={3–4}, B3={5}, B4={6–7}. Arestas: B1→B3 por salto verdadeiro; B1→B2 por continuação; B2→B4 por goto; B3→B4 por continuação.

3. Otimizações locais

Otimização local examina um bloco básico isoladamente, embora algumas decisões — sobretudo se um valor calculado é necessário na saída — possam se beneficiar de informação global de liveness. Um grafo acíclico dirigido (DAG) pode representar valores calculados no bloco: nós representam valores iniciais, operações e definições; rótulos associam variáveis ao valor corrente. Valores iniciais distintos, como x_0 e x_1, não devem ser confundidos.

3.1 Simplificação algébrica e folding de constantes

Simplificação algébrica substitui uma operação por outra equivalente segundo a semântica e os tipos da linguagem. Constant folding avalia expressões compostas somente de constantes durante a compilação. Exemplos típicos, sujeitos às regras da implementação: x+0→x; x−0→x; x×1→x; x/1→x; 3×4→12; -(2+5)→-7. Divisão por zero constante é diagnóstico, não uma simplificação válida.

Antes	Depois	Condição/observação
t1 = 3 * 4	t1 = 12	Tipos e overflow compatíveis
t2 = x + 0	t2 = x	A soma não tem efeito colateral
t3 = y * 1	t3 = y	Tipos numéricos válidos
t4 = z / 0	Erro semântico/compilação	A especificação MINIC exige rejeitar divisor constante zero

Transformações como x*0→0 exigem cautela: no modelo matemático usual são verdadeiras, mas uma implementação pode definir overflow, exceções, NaN ou efeitos observáveis. Além disso, eliminar a avaliação de uma expressão que possa falhar ou produzir efeito colateral não é automaticamente seguro. A regra do compilador deve ser escrita para a semântica efetiva de MINIC, não apenas para a álgebra escolar.

3.2 Propagação local de constantes

A propagação local mantém, dentro do bloco, um mapa de valores conhecidos. Ao encontrar x=constante, registra-se x↦constante; em seguida, substituem-se usos de x enquanto essa definição permanecer válida. Uma nova atribuição a x invalida a associação anterior. Após substituição, expressões constantes podem ser avaliadas.

O exemplo é local porque não cruza fronteiras de bloco. Na presença de junções de controle, saber que x vale uma constante em um predecessor não basta: o valor pode diferir em outro predecessor. Essa é a motivação para a propagação global de constantes.

3.3 Eliminação de cópias

Na cópia x=y, pode-se substituir usos posteriores de x por y enquanto nenhuma atribuição a x ou y invalide a relação e enquanto não houver questão de tipo, escopo ou alias. A cópia pode então ser removida se x não tiver outro uso. A análise de cópias costuma ser seguida da eliminação de código morto.

Uma chamada de função, uma escrita indireta ou um acesso a vetor podem alterar valores acessíveis. Sem análise de efeitos/alias, o otimizador deve invalidar conservadoramente fatos que possam ter sido afetados. O MINIC mínimo não exige ponteiros, mas chamadas, vetores e variáveis globais ainda exigem atenção a efeitos.

3.4 Eliminação de código morto

Uma definição é morta se seu valor não puder ser usado em nenhum caminho futuro antes de ser substituído e não produzir efeito observável. Uma atribuição a variável local cujo resultado nunca é lido pode ser removida; print, read, chamada com efeitos e comandos que podem falhar não são descartáveis só por não produzirem um valor usado. Em um DAG, remove-se repetidamente uma raiz sem variável viva associada; isso pode tornar seus operandos também dispensáveis.

A classificação de “vivo na saída” não pode ser escolhida arbitrariamente. Se x é usado em outro bloco, seu valor deve ser preservado. Por isso, a eliminação local mais segura usa liveness global ou adota uma hipótese conservadora que considera vivas as variáveis que podem escapar do bloco.

3.5 Eliminação local de subexpressões comuns

Uma expressão é subexpressão comum local quando já foi computada no bloco e os valores de seus operandos não mudaram desde então. Em um DAG, expressões com o mesmo operador e os mesmos filhos (na mesma ordem quando necessário) podem compartilhar um nó.

A igualdade textual não basta. Em a[i]; a[j]=v; x=a[i], a última leitura não pode ser substituída automaticamente pela primeira, pois i pode ser igual a j. Uma escrita em vetor invalida fatos sobre elementos que possam ter sido modificados. Em MINIC, tratar STORE/LOAD de vetores com alias conservador evita otimização incorreta.

4. Análise de fluxo de dados: estrutura comum

Análise de fluxo de dados calcula fatos abstratos em pontos do programa a partir da IR e do grafo de fluxo. O domínio descreve a informação que interessa à análise: conjuntos de definições, conjuntos de expressões ou conjuntos de variáveis. Para cada bloco B, IN[B] é o fato na entrada e OUT[B] na saída. A função de transferência fB descreve como o bloco transforma a informação: OUT[B]=fB(IN[B]) para análises forward.

Uma análise forward propaga informação no sentido do fluxo de execução, de predecessores para sucessores. Uma análise backward propaga informação do futuro para o passado, de sucessores para predecessores. “Forward/backward” descreve direção, não se a operação usa união ou interseção; o meet depende da pergunta semântica.

4.1 Funções gen/kill

Em muitos problemas baseados em conjuntos, um bloco gera fatos (gen) e invalida outros (kill). A função tem a forma fB(X)=genB ∪ (X − killB). genB contém fatos produzidos pelo bloco e ainda válidos ao sair; killB contém fatos anteriores que deixam de ser válidos. A composição das funções das instruções, na ordem do bloco, produz a função do bloco. Para problemas com interseção ou domínios de valores, a função pode ter outra forma.

4.2 Equações de fluxo

Para uma análise forward, IN[B] é obtido combinando OUT dos predecessores; então aplica-se fB para obter OUT[B]. Para uma análise backward, OUT[B] combina IN dos sucessores; aplica-se a função do bloco no sentido inverso, obtendo IN[B]. Em análise por conjuntos, o operador de combinação (meet) pode ser união ou interseção.

A solução é encontrada por aproximação sucessiva: inicializam-se os conjuntos conforme o problema e as condições de contorno; reavaliam-se as equações até que nenhum IN/OUT mude (ponto fixo). Como o grafo pode conter ciclos, uma única passagem geralmente não basta. O domínio finito e a monotonicidade das funções sob hipóteses apropriadas garantem a convergência do algoritmo iterativo.

5. Reaching definitions — definições que alcançam

Uma definição d de x alcança um ponto p se existe um caminho da definição até p sem outra definição de x no caminho. O resultado é um conjunto de definições possíveis, e não um valor único. É uma análise forward com união: uma definição pode alcançar o ponto por qualquer predecessor. A informação pode apoiar propagação de cópias, diagnóstico de uso antes de definição e outras otimizações.

Para cada bloco B, genB contém as definições do bloco que permanecem visíveis na saída; killB contém as demais definições das variáveis definidas no bloco. Equações: IN[B]=∪ OUT[P]

para P predecessor; OUT[B]=genB ∪ (IN[B]−killB). No ENTRY, OUT[ENTRY]=∅ para o exemplo sem definições fictícias de parâmetros/entrada.

5.1 Exemplo numérico de reaching definitions

Arestas: B1→B2, B1→B3, B2→B4, B3→B4. Universo D={d1,d2,d3}. B1 gera {d1}; B2 gera {d2} e mata {d1}; B3 gera {d3} e não mata definição de x; B4 não altera definições. Com a inicialização ∅: OUT[B1]={d1}; IN[B2]={d1}, OUT[B2]={d2}; IN[B3]={d1}, OUT[B3]={d1,d3}; IN[B4]=OUT[B2]∪OUT[B3]={d1,d2,d3}. Na linha print(x), d1 ou d2 podem ser a última definição. d3 define y, portanto não interfere com x.

A análise pode indicar que uma definição é possível, não que todo caminho a contém. Se for necessária uma propriedade verdadeira em todos os caminhos, a escolha do meet e do domínio deve refletir isso. Essa distinção é essencial para não promover uma otimização baseada em um caminho apenas.

6. Available expressions — expressões disponíveis

Uma expressão é disponível em um ponto se foi calculada em todo caminho desde a entrada relevante e nenhum operando foi redefinido depois do cálculo, antes desse ponto. Isso permite reutilizar uma computação em vez de recalculá-la. A análise é forward e usa interseção na junção: uma expressão só está disponível se estiver disponível ao fim de todos os predecessores.

Para expressões elegíveis, e_genB reúne as expressões avaliadas no bloco e ainda válidas na saída; e_killB reúne expressões invalidadas por atribuição a algum operando. Equações: IN[B]=∩ OUT[P]; OUT[B]=e_genB ∪ (IN[B]−e_killB). A condição de entrada é vazia: nenhuma expressão foi computada antes do início do procedimento.

6.1 Exemplo de junção

B1 gera a+b. O caminho B1→B2 redefine a; portanto, a+b não chega disponível a B4 por B2. A interseção dos OUT dos predecessores (B2 e B3) não contém a+b, então t2=a+b não pode ser substituído por t1 sem outra prova. A existência do cálculo no predecessor B1 não basta: é preciso preservar os operandos em todos os caminhos.

Quando B2 não redefine a nem b, a+b pode permanecer disponível em ambos os caminhos e a expressão em B4 pode ser redundante. Ainda é necessário garantir que as ocorrências representem a mesma operação tipada e que a avaliação não tenha efeitos laterais ou possibilidade de erro com comportamento observável.

7. Live variables — variáveis vivas

Uma variável x está viva em um ponto se o valor corrente de x pode ser usado em algum caminho futuro antes de receber nova definição. Está morta se esse valor não puder mais influenciar o comportamento observável. É análise backward; sua informação ajuda a eliminar código morto e a decidir quais valores precisam ser preservados entre blocos ou em registradores.

Para cada bloco B, useB contém variáveis usadas antes de serem definidas no bloco; defB contém variáveis definidas antes de qualquer uso anterior delas. Equações: OUT[B]=∪ IN[S] para S sucessor; IN[B]=useB ∪ (OUT[B]−defB). Para o EXIT, IN[EXIT]=∅ se não há usos posteriores ao fim do procedimento; saídas observáveis e retornos devem ser modelados como usos.

7.1 Cálculo manual

Ignorando c como valor recebido fora do fragmento, B1 define a e usa c; para o exemplo, suponha que a condição use uma entrada já válida. B2 usa a e define b: use={a}, def={b}. B3 define b: use=∅, def={b}. B4 usa b: use={b}, def=∅. Em B4, IN={b}. Logo B2 e B3 têm OUT contendo b; como definem b, IN[B2]={a} e IN[B3]=∅. Assim, o valor de a é necessário na entrada de B2, mas não de B3.

A definição de “variável viva” refere-se ao valor, não somente ao nome. Se x é redefinida antes do próximo uso, o valor antigo de x está morto mesmo que o nome x volte a aparecer. Essa distinção evita tanto manter valores desnecessários quanto eliminar definições necessárias.

8. Propagação global de constantes

A propagação global de constantes combina informação através dos blocos. Para cada variável, o domínio abstrato pode registrar uma constante c, NAC (“não é constante”) ou UNDEF (“ainda sem definição observada”). Uma variável só pode ser substituída por c se a análise provar que tem esse mesmo valor em todos os caminhos que chegam ao ponto. Constantes diferentes na junção produzem NAC.

Valores combinados	Resultado típico	Leitura
c e c	c	A mesma constante em todos os caminhos
c1 e c2, c1≠c2	NAC	Pode assumir valores distintos
UNDEF e c	c	Uma entrada ainda não acrescenta alternativa
NAC e c	NAC	Já não há garantia de constância
NAC e c	NAC	Já não há garantia de constância

A propagação é forward. A função de transferência atualiza o estado abstrato conforme cada instrução: x=c associa c; x=y copia a informação de y; x=y op z calcula c quando os operandos são constantes e a operação é definida; caso contrário, x torna-se NAC. A junção dos predecessores combina, componente a componente, os valores abstratos. A análise itera até estabilizar.

8.1 Exemplo de junção de constantes

Nas duas saídas, x=5; portanto, na junção x continua constante 5 e y pode ser substituída por 7. Se o ramo else atribuísse x=8, a junção resultaria em NAC; y=x+2 não poderia ser reduzida a uma constante única. Em MINIC, o tipo de x e a validade da operação precisam ser compatíveis.

A especificação do livro-texto destaca uma sutileza: a estrutura de propagação de constantes não é simplesmente uma instância distributiva trivial como alguns esquemas de conjuntos; seu

domínio e sua junção devem distinguir UNDEF de NAC. Na prática didática, o princípio operacional é manter fatos conservadores e não concluir constância quando caminhos discordam.

9. Relação entre as análises e as otimizações

Análise	Direção	Domínio/meet	Uso típico
Reaching definitions	Forward	Conjuntos de definições; união	Saber quais atribuições podem fornecer um valor
Available expressions	Forward	Conjuntos de expressões; interseção	Eliminar subexpressões comuns globais
Live variables	Backward	Conjuntos de variáveis; união	Eliminar definições mortas; apoiar alocação
Propagação de constantes	Forward	Mapeamentos de valores; junção abstrata	Substituir valores provadamente constantes

Essas análises se complementam. A disponibilidade de uma expressão não é igual a reaching definitions; uma variável viva não é necessariamente constante; e uma definição que alcança um ponto pode ser uma entre várias alternativas. O otimizador precisa casar cada transformação com a análise apropriada e conservar o programa quando a informação não é conclusiva.

10. Processo prático de otimização MINIC

1. Validar primeiro: o programa precisa passar por lexer, parser, AST e análise semântica; não se otimiza código inválido como se fosse executável.
2. Gerar IR tipada, com temporários e rótulos explícitos; associar cada instrução à origem na AST quando possível.
3. Particionar por procedimento em blocos básicos e construir sucessores/predecessores a partir dos saltos.
4. Executar passes locais: folding, simplificação, cópias, subexpressões comuns e código morto, revalidando efeitos e tipos.
5. Executar análises globais selecionadas; inicializar condições de contorno e iterar até ponto fixo.
6. Aplicar transformação apenas quando os fatos a justificam; atualizar ou recalcular análises afetadas.
7. Comparar execução de O0 e O1 em casos válidos e inválidos, incluindo caminhos alternativos, ciclos, vetores e efeitos observáveis.

10.1 Invariantes de corretude para testes

- O código otimizado conserva a saída observável e o código de saída do programa nos casos válidos testados.
- Erros semânticos definidos pela especificação — como incompatibilidade de tipos ou divisor constante zero — continuam sendo rejeitados.

- A transformação não remove print/read, chamada com efeito, retorno, salto ou operação cuja falha seja observável.
- Testes de junção exercitam caminhos com valores iguais e diferentes; testes de laço exercitam múltiplas iterações e saída imediata.
- A IR otimizada continua bem formada: todos os rótulos existem, temporários são definidos antes de uso quando exigido, e tipos permanecem coerentes.

11. Exercícios

Exercício 1 — blocos e fluxo

Para a sequência do item 2.3, identifique os líderes, partições e arestas. Explique por que a instrução após um salto condicional começa um novo bloco.

Exercício 2 — otimizações locais

Otimize, sempre que seguro, e indique a razão: (a) t1=2*5; t2=t1+0; x=t2; print(x). (b) q=a+b; r=a+b; print(r). (c) x=1; x=2; print(x). (d) t=a[i]; a[j]=9; u=a[i].

Exercício 3 — reaching definitions

No grafo B1→B2, B1→B3, B2→B4, B3→B4: B1 contém d1:x=1; B2 contém d2:x=2; B3 não define x; B4 usa x. Calcule IN e OUT relevantes e diga quais definições alcançam o uso em B4.

Exercício 4 — live variables

Considere B1: a=b+c; B2: print(a); return 0. Calcule use/def, IN e OUT para cada bloco, assumindo B1→B2 e EXIT após B2. Qual valor precisa sobreviver de B1 para B2?

Exercício 5 — expressões disponíveis e junções

Em dois ramos que convergem, ambos calculam p+q sem redefinir p ou q. A expressão p+q está disponível após a junção? E se um ramo redefine q antes da junção? Justifique pelo meet da análise.

Exercício 6 — propagação global de constantes

Analise: if (c) x=6; else x=6; y=x*2. Repita alterando a atribuição do else para x=7. Em qual caso y pode ser reduzida a constante? Que fato abstrato aparece na junção?

12. Respostas comentadas

1.

Líderes: 1, 3, 5 e 6; blocos {1–2}, {3–4}, {5}, {6–7}. O sucessor de um salto condicional é um ponto possível de entrada por fluxo que não executou a instrução seguinte pelo caminho sequencial; por isso, a instrução seguinte não pode permanecer no mesmo bloco.

2.

(a) t1=10, t2=10, x=10, print(10), se os tipos e a semântica da operação permitirem folding. (b) r pode reutilizar q, pois a+b não foi invalidada: r=q; em seguida q pode ser removida se não tiver

uso. (c) x=1 é morta se nenhum efeito depender dela; manter x=2 e print(x). (d) não substituir u por t sem prova de que a escrita a[j] não alterou a[i]; j pode ser igual a i.

3.

OUT[B1]={d1}. IN[B2]={d1}; OUT[B2]={d2}, pois d2 mata d1 para x. IN[B3]={d1}; OUT[B3]={d1}. IN[B4]=OUT[B2]∪OUT[B3]={d1,d2}. Ambas podem alcançar o uso de x.

4.

B1: use={b,c}, def={a}; B2: use={a}, def=∅. Para EXIT, IN=∅; portanto OUT[B2]=∅ e IN[B2]={a}. Logo OUT[B1]={a} e IN[B1]={b,c}. O valor de a precisa sobreviver à fronteira B1→B2; b e c são necessários antes de B1 definir a.

5.

Se ambos os ramos calculam p+q e não redefinem os operandos depois do cálculo, a expressão pertence ao OUT de ambos e à interseção na junção. Se um ramo redefine q sem recalcular p+q, a expressão não está disponível em todos os caminhos e sai da interseção.

6.

Com x=6 em ambos os ramos, a junção preserva x↦6; y pode virar 12, respeitadas as regras de tipo. Se um ramo atribui 7, a junção combina constantes distintas em NAC; y não pode ser reduzida a um literal único.

Síntese

Blocos básicos isolam sequências sem desvios internos; o grafo de fluxo conecta esses blocos e revela junções e ciclos. Otimizações locais exploram valores e redundâncias dentro do bloco, enquanto análises forward e backward calculam fatos globais em todos os caminhos. A decisão segura exige semântica preservada, modelos conservadores para efeitos e uma análise compatível com a transformação.

Referências e alinhamento curricular

AHO, Alfred V.; LAM, Monica S.; SETHI, Ravi; ULLMAN, Jeffrey D. Compiladores: princípios, técnicas e ferramentas. 2. ed. São Paulo: Pearson/Addison-Wesley, 2008. Cap. 8, §§ 8.4–8.5; Cap. 9, §§ 9.1–9.4. (A paginação e a grafia podem variar conforme a edição.)

ESPECIFICAÇÃO DA LINGUAGEM MINIC. Versão 1.0 — Documento acadêmico. Seções 5–10: tipos e conversões, escopos, semântica, AST, IR e requisitos de otimização.